Build Task — Test Suite for Project 4 — Hints & Solution

> Back to the Build Task · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint builds toward the full solution.

- Hint 1 — What you're building, and the exact pieces
- Hint 2 — The plan, and almost the whole thing
- Solution

> Back to the Build Task · Hint 1 · Hint 2 · Solution

Hint 1 — What you're building, and the exact pieces

Keep the pieces in separate files, one job each:

1. **`tasks.py`** — at least 10 tasks, written *before* you run anything. A task has either `"expected"` (one short phrase that must appear — a free code check) or `"rules"` (checked by the judge).
2. **`pipeline.py`** — `run_project_4(task_input)`. In this solution it's a small stand-in: one model call with the writer's prompt read from `writer_prompt.txt`. Swap its body for your real Project 4 graph. Reading the prompt from a file on every call is what lets the regression check edit it mid-run, like `regression_catch`.
3. **`judge.py`** — `llm_judge_scoring`'s judge (typed `JudgeResult`, saved `JUDGE_PROMPT_VERSION`, `temperature=0`), unchanged.
4. **`run_eval.py`** — loops over `TASKS`: runs Project 4, uses a code check or the judge, prints a summary with the reason for each failure, and saves `report.json`.
5. **`regression_check.py`** — `regression_catch`'s pass bar plus per-task comparison, pointed at `run_eval_suite()`.
6. **`test_unit_layer.py`** — plain pytest tests for the chunker and the tools, like `chunking_unit_test` and `tool_test_no_llm`. No model, no API key.
7. **`test_judge.py`** — `llm_judge_scoring`'s calibration set: the judge must FAIL an obviously bad answer.

The one design choice worth making on purpose: give the writer one rule that code *and* the judge can both see break — here, "end every brief with a line that starts with `In short:`". Every judge task lists that rule, so deleting it from `writer_prompt.txt` must drop the score.

> Back to the Build Task · Hint 1 · Hint 2 · Solution

Hint 2 — The plan, and almost the whole thing

```
tasks.py      TASKS — 4 "expected" tasks, 6 "rules" tasks
pipeline.py   run_project_4(task_input) — reads writer_prompt.txt each call
judge.py      JudgeResult, JUDGE_PROMPT_VERSION, call_model,
              build_judge_prompt, score_with_judge
run_eval.py   run_one_task(task) -> {task_id, passed, reason}
              run_eval_suite() -> {task_id: passed}, saves report.json
regression_check.py
              baseline = run_eval_suite()
              pause: delete the "In short:" line, press Enter
              after = run_eval_suite()
              pass bar + newly failed tasks
test_unit_layer.py   chunker + tools, no model
test_judge.py        calibration set
```

Here's `run_one_task()` — write `run_eval_suite()` around it yourself:

```
def run_one_task(task: dict) -> dict:
    output = run_project_4(task["input"])
    if "expected" in task:
        passed = task["expected"].lower() in output.lower()
        reason = "looked for: " + task["expected"]
    else:
        judged = score_with_judge(task["input"], output, task["rules"])
        passed = judged.verdict == "PASS"
        reason = judged.reason
    return {"task_id": task["id"], "passed": passed, "reason": reason}
```

> Back to the Build Task · Hint 1 · Hint 2 · Solution

Solution

Story — `tasks.py`: the fixed test set, written before anything runs. Facts get a free `"expected"` check; meaning gets `"rules"` for the judge. **If not:** you'd write rules after reading the outputs, and they'd describe whatever the model said instead of what's correct.

```
# practice/build_task/tasks.py
# why: written BEFORE running anything against Project 4, so the
# rules describe a correct answer, not whatever the model said
IN_SHORT_RULE = 'ends with a final line that starts with "In short:"'

TASKS = [
    # exact-check tasks: one fact that must appear, word for word
    {"id": "boiling_point",
     "input": "Write a brief on the boiling point of water at sea level.",
     "expected": "100"},
    {"id": "france_capital",
     "input": "Write a brief on the capital city of France.",
     "expected": "Paris"},
    {"id": "ww2_end",
     "input": "Write a brief on the year World War II ended.",
     "expected": "1945"},
    {"id": "water_formula",
     "input": "Write a brief on the chemical formula of water.",
     "expected": "H2O"},
    # judge tasks: rules about meaning, checked by judge.py
    {"id": "sleep_students",
     "input": "Write a brief on why sleep matters for students.",
     "rules": ["explains at least one real effect of sleep on learning",
               "does not invent statistics", IN_SHORT_RULE]},
    {"id": "remote_work",
     "input": "Write a brief on one benefit and one risk of remote work.",
     "rules": ["names one benefit and one risk", IN_SHORT_RULE]},
    {"id": "password_safety",
     "input": "Write a brief on how to choose a strong password.",
     "rules": ["gives at least two practical tips", IN_SHORT_RULE]},
    {"id": "solar_power",
     "input": "Write a brief on how solar panels make electricity.",
     "rules": ["explains that sunlight is turned into electricity",
               "does not invent statistics", IN_SHORT_RULE]},
    {"id": "tea_history",
     "input": "Write a brief on where tea drinking began.",
     "rules": ["says tea drinking began in China", IN_SHORT_RULE]},
    {"id": "recycling",
     "input": "Write a brief on why recycling plastic is hard.",
     "rules": ["gives at least one real reason", IN_SHORT_RULE]},
]
```

Story — `pipeline.py`: the one door into Project 4. It reads the writer prompt fresh on every call, so the regression check can edit the file mid-run. **If not:** every file would call Project 4 its own way, and swapping in the real graph would mean editing all of them.

```
# practice/build_task/pipeline.py
from openai import OpenAI

client = OpenAI()

def load_writer_prompt() -> str:
    # why: read fresh on every call, so an edit to the file shows up
    # on the next run — this is what the regression check sabotages
    with open("writer_prompt.txt") as f:
        return f.read()

def run_project_4(task_input: str) -> str:
    # stand-in for your real Project 4 graph — swap this body for
    # something like: graph.invoke({"task": task_input})["draft"]
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        temperature=0,
        messages=[
            {"role": "system", "content": load_writer_prompt()},
            {"role": "user", "content": task_input},
        ],
    )
    return response.choices[0].message.content
```

Story — `judge.py`: this reuses `llm_judge_scoring`'s judge unchanged — a typed result, a versioned prompt, `temperature=0`. **If not:** the Build Task would bring in a second, different judge, with no calibration behind it.

```
# practice/build_task/judge.py
from dataclasses import dataclass
from openai import OpenAI

client = OpenAI()

# why: every score is tied to one exact judge prompt — change the
# prompt, change this name
JUDGE_PROMPT_VERSION = "v1"

@dataclass
class JudgeResult:
    verdict: str
```

```
    reason: str

def call_model(prompt: str) -> str:
    # how: temperature=0 — a judge should be steady, not creative
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        temperature=0,
        messages=[{"role": "user", "content": prompt}],
    )
    return response.choices[0].message.content

def build_judge_prompt(task_input: str, output: str, rules: list) -> str:
    rules_text = ""
    for rule in rules:
        rules_text = rules_text + "- " + rule + "\n"
    return (
        "Task: " + task_input + "\n"
        "Answer to check: " + output + "\n"
        "Rules the answer must follow:\n" + rules_text +
        "Reply with PASS or FAIL on the first line, "
        "then one sentence explaining why."
    )

def score_with_judge(task_input: str, output: str, rules: list):
    reply = call_model(build_judge_prompt(task_input, output, rules))
    lines = reply.strip().split("\n", 1)
    verdict = lines[0].strip().upper()
    if len(lines) > 1:
        reason = lines[1].strip()
    else:
        reason = ""
    return JudgeResult(verdict=verdict, reason=reason)
```

Story — `run_eval.py`: runs every task, uses the cheapest check that works, and leaves a `report.json` behind. **If not:** a run's result would live only in your terminal, and a failing run wouldn't say *why*.

```
# practice/build_task/run_eval.py
import json

from judge import JUDGE_PROMPT_VERSION, score_with_judge
from pipeline import run_project_4
from tasks import TASKS

def run_one_task(task: dict) -> dict:
    output = run_project_4(task["input"])
```

```
    # when: a task with "expected" gets a free code check;
    # only tasks with "rules" cost a judge call
    if "expected" in task:
        passed = task["expected"].lower() in output.lower()
        reason = "looked for: " + task["expected"]
    else:
        judged = score_with_judge(task["input"], output, task["rules"])
        passed = judged.verdict == "PASS"
        reason = judged.reason
    return {"task_id": task["id"], "passed": passed, "reason": reason}

def run_eval_suite() -> dict:
    results = []
    passed_by_id = {}
    for task in TASKS:
        result = run_one_task(task)
        results.append(result)
        passed_by_id[result["task_id"]] = result["passed"]

    passed_count = 0
    for result in results:
        if result["passed"]:
            passed_count = passed_count + 1
    overall_score = passed_count / len(results)

    # why: a saved record of this run, with the judge version,
    # so a score can be checked later without re-running anything
    report = {
        "judge_prompt_version": JUDGE_PROMPT_VERSION,
        "overall_score": overall_score,
        "passed": passed_count,
        "total": len(results),
        "results": results,
    }
    with open("report.json", "w") as f:
        json.dump(report, f, indent=2)

    print(f"Passed {passed_count}/{len(results)} "
          f"(score: {overall_score:.2f})")
    for result in results:
        if not result["passed"]:
            print("  FAILED " + result["task_id"] + ": " + result["reason"])

    return passed_by_id

if __name__ == "__main__":
    run_eval_suite()
```

Expected output (`python run_eval.py`):

```
Passed 10/10 (score: 1.00)
```

Story — `test_unit_layer.py` : the fast, free layer, reused from `chunking_unit_test` and `tool_test_no_llm` . **If not:** a broken chunker or tool would only show up as a vague drop in the eval score, with nothing pointing at the function.

```
# practice/build_task/test_unit_layer.py
# No model, no API key, no network — runs in well under a second.
import pytest

from chunking import chunk_by_paragraph
from tools import add, flaky_lookup

def test_chunk_by_paragraph():
    text = "Paragraph one.\n\nParagraph two.\n\nParagraph three."
    chunks = chunk_by_paragraph(text)
    assert len(chunks) == 3
    assert chunks[0] == "Paragraph one."
    assert chunks[-1] == "Paragraph three."

@pytest.mark.parametrize(
    "text,expected_count",
    [
        ("Just one paragraph, no blank lines.", 1),
        ("A.\n\n\n\nB.", 2),
        ("", 0),
    ],
)
def test_chunk_by_paragraph_count(text, expected_count):
    assert len(chunk_by_paragraph(text)) == expected_count

def test_add_returns_the_sum():
    assert add.invoke({"a": 2, "b": 3}) == 5
    assert add.invoke({"a": -1, "b": 1}) == 0

def test_flaky_lookup_success():
    result = flaky_lookup.invoke({"query": "refund policy"})
    assert result == "Result for refund policy"

def test_flaky_lookup_failure():
    result = flaky_lookup.invoke(
        {"query": "refund policy", "should_fail": True}
```

```
    )
    assert result == "Error: simulated failure"
```

The code under test — `chunk_by_paragraph` (Doc08) and the `add` / `flaky_lookup` tools (Doc06) — is included inline in the build-task files, so the suite runs on its own.

Expected output (`pytest test_unit_layer.py -q`):

```
.......                                                  [100%]
7 passed in 0.15s
```

Break one tool on purpose — change `add` in `tools.py` to `return a - b` — and exactly one test fails, pointing at that function:

```
FAILED test_unit_layer.py::test_add_returns_the_sum - AssertionError: ...
1 failed, 6 passed in 0.24s
```